

INTERRUPTS & INTERRUPT HANDLING/ (PROCEDURE CALLS)

Interrupts

Mechanism by which other modules (e.g. I/O) or conditions may interrupt normal sequence of processing

Program

- e.g. overflow, division by zero

Timer

- Generated by internal processor timer
- Used in pre-emptive multi-tasking

I/O

- from I/O controller

Hardware failure

- e.g. memory parity error

The figure consists of three panels, each showing a timeline for a User Program and an I/O Program. The User Program has three 'WRITE' operations labeled 1, 2, and 3. The I/O Program has an 'I/O Command' labeled 4 and an 'END' event labeled 5. Dotted lines represent the execution flow between these events.

- Panel 1 (Left): Non-preemptive interrupt handling.** The User Program starts with WRITE 1. An interrupt occurs (indicated by a vertical dashed line). The I/O Program starts its I/O Command (4). The User Program continues to WRITE 2 and then WRITE 3. The I/O Program finishes its I/O Command (4) and reaches END (5). The I/O Program's execution is delayed because it must wait for the User Program to finish its current WRITE operation.
- Panel 2 (Middle): Preemptive interrupt handling.** The User Program starts with WRITE 1. An interrupt occurs. The I/O Program starts its I/O Command (4). The User Program continues to WRITE 2. An interrupt occurs again. The I/O Program finishes its I/O Command (4) and reaches END (5). The User Program then continues to WRITE 3. The I/O Program's execution is delayed because it must wait for the User Program to finish its current WRITE operation.
- Panel 3 (Right): Priority-based interrupt handling.** The User Program starts with WRITE 1. An interrupt occurs. The I/O Program starts its I/O Command (4). The User Program continues to WRITE 2. An interrupt occurs again. The I/O Program finishes its I/O Command (4) and reaches END (5). The User Program then continues to WRITE 3. The I/O Program's execution is delayed because it must wait for the User Program to finish its current WRITE operation.

(c) Interrupts; long I/O wait

Interrupt Cycle

At end of each instruction cycle

Processor checks for interrupt

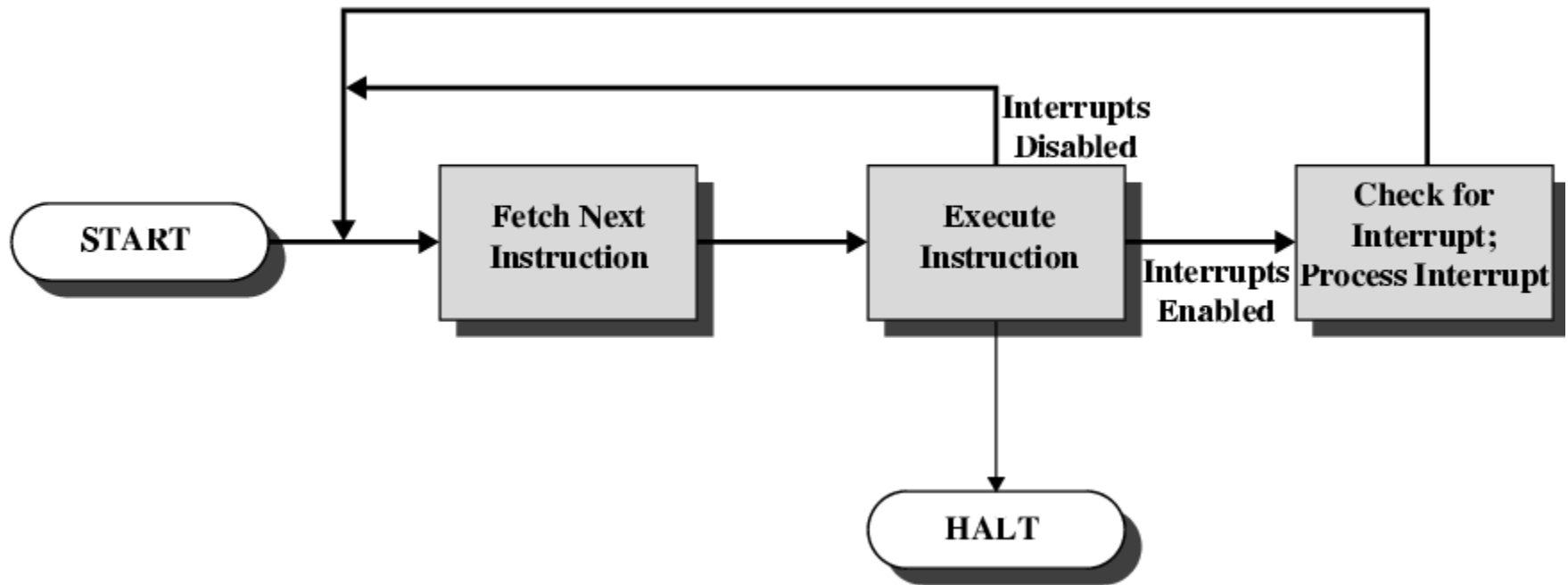
- Indicated by an interrupt signal

If no interrupt, fetch next instruction

If interrupt pending:

- Suspend execution of current program
- Save context
- Set PC to start address of interrupt handler routine
- Process interrupt
- Restore context and continue interrupted program

Instruction Cycle with Interrupts

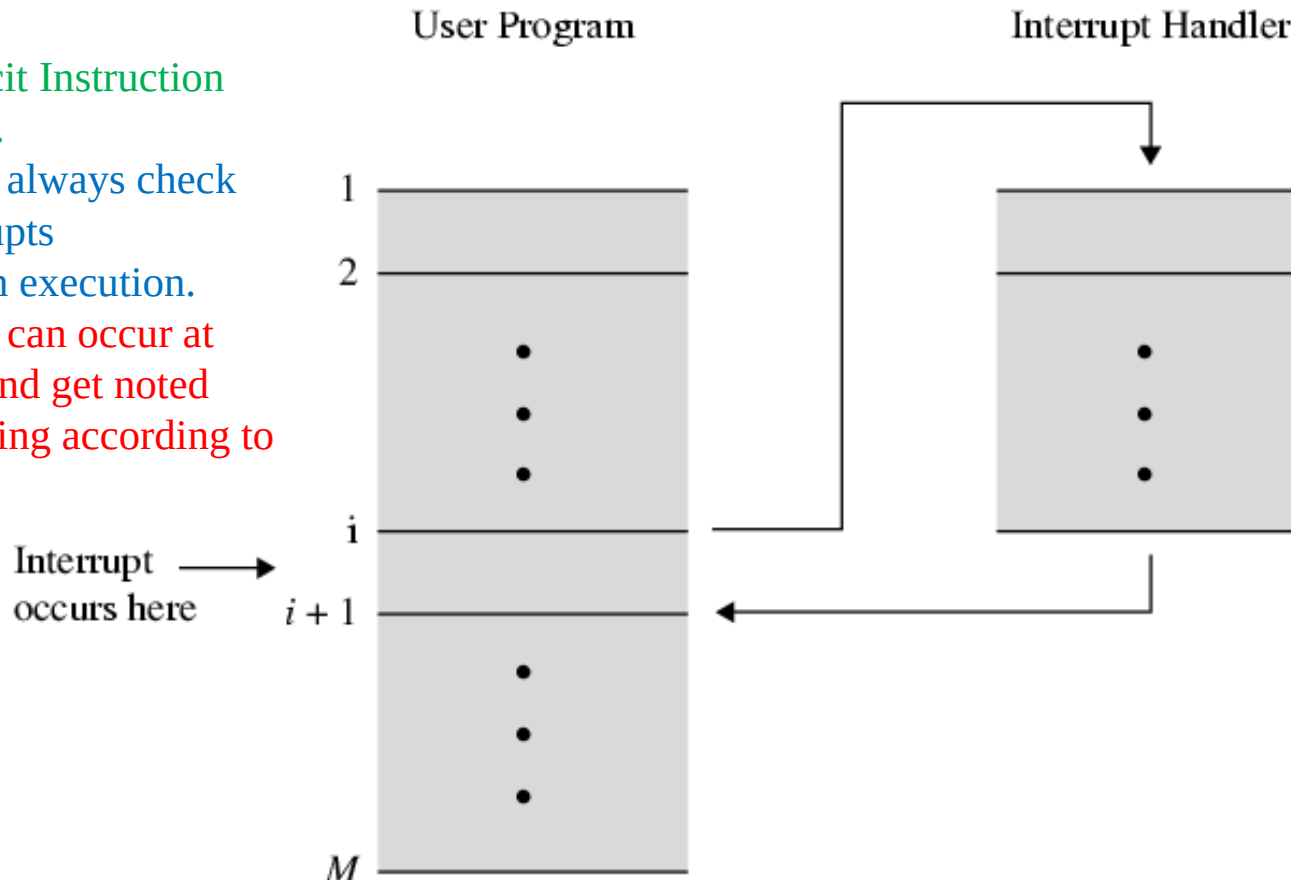


Transfer of Program Control to Interrupts Handler Routine

No Explicit Instruction
for Jump.

Processor always check
for interrupts
after each execution.

Interrupts can occur at
anytime and get noted
for handling according to
priority.



Handling Multiple Interrupts

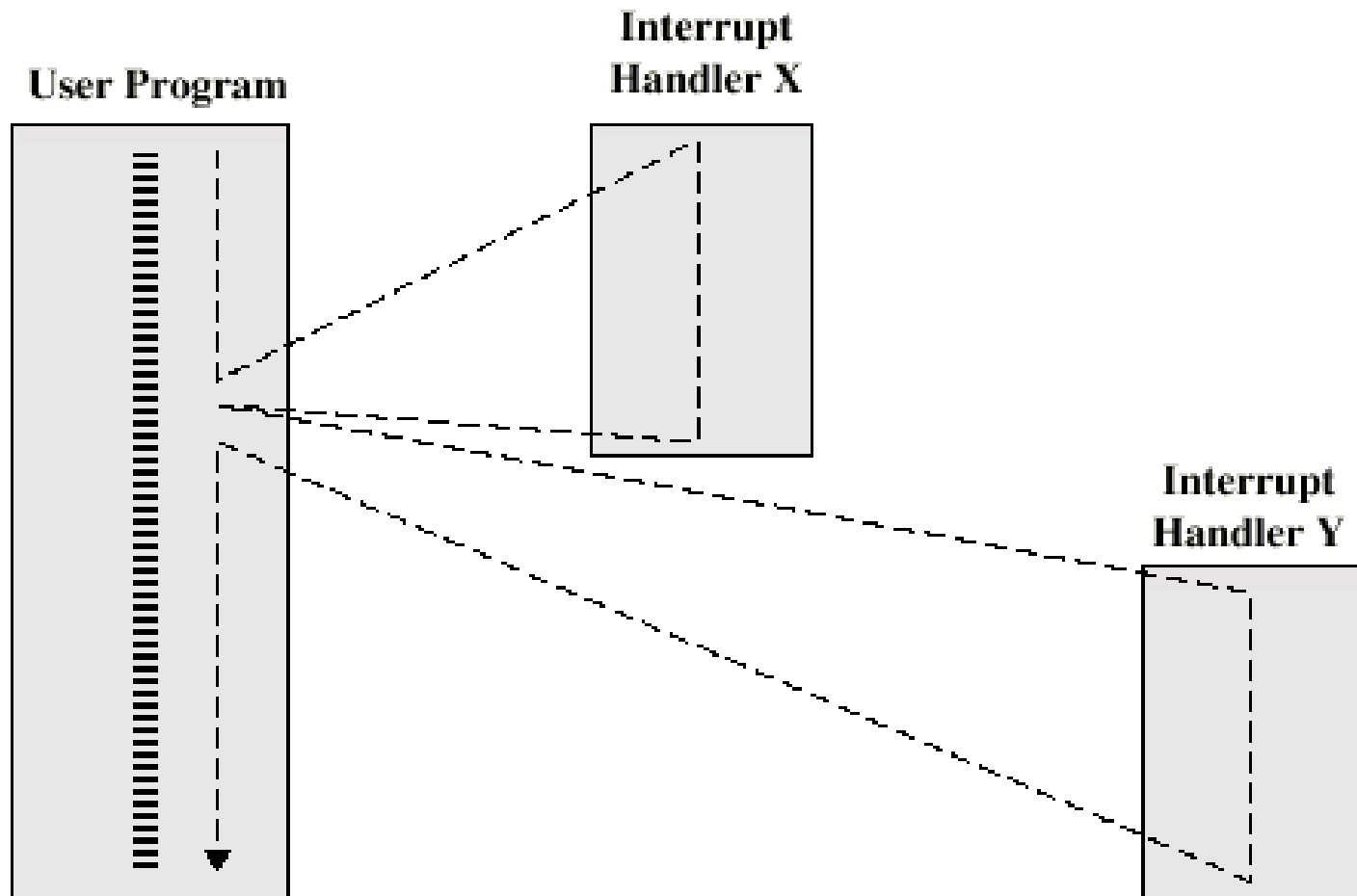
Disable interrupts (during an interrupt handling)

- Processor will not act on further interrupts whilst processing one interrupt
- Interrupts remain pending and are checked after first interrupt has been processed
- Interrupts handled in sequence as they occur

Define priorities (priority interrupt first)

- Low priority interrupts can be interrupted by higher priority interrupts
- When higher priority interrupt has been processed, processor returns to previous interrupt

Multiple Interrupts - Sequential



Multiple Interrupts - Sequential

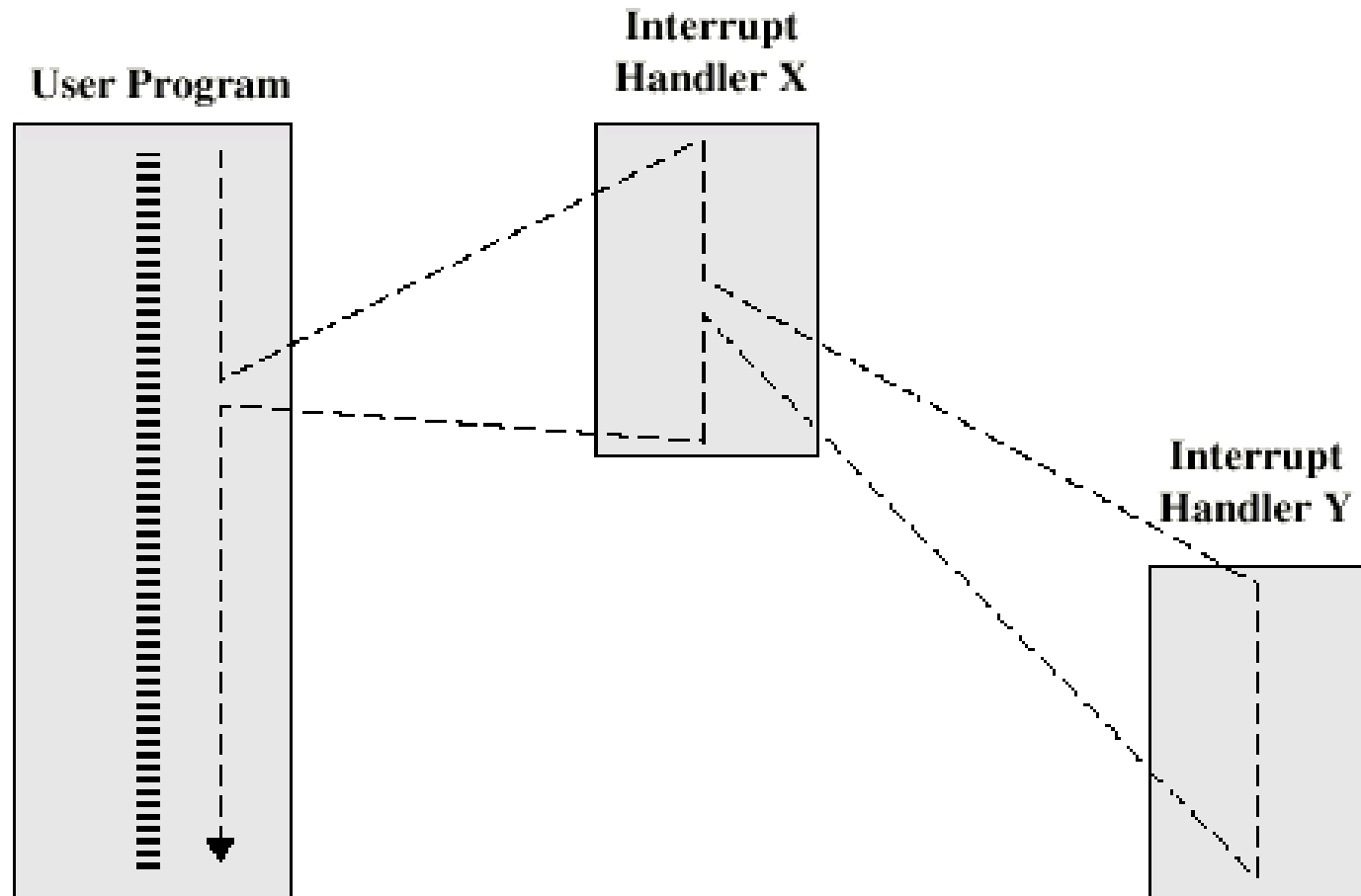
The drawback to the preceding approach is that it does not take into account relative priority or time-critical needs.

For example, when input arrives from the communications line, it may need to be absorbed rapidly to make room for more input. If the first batch of input has not been processed before the second batch arrives, data may be lost.

Multiple Interrupts – Nested

A second approach is to define priorities for interrupts and to allow an interrupt of higher priority to cause a lower-priority interrupt handler to be itself interrupted

Multiple Interrupts – Nested



Multiple Interrupts: Example

Consider a system with three I/O devices: a printer, a disk, and a communications line, with increasing priorities of 2, 4, and 5, respectively.

A user program begins at $t=0$. At $t=10$, a printer interrupt occurs; user information is placed on the system stack and execution continues at the printer interrupt service routine (ISR).

While this routine is still executing, at $t=15$, a communications interrupt occurs. Because the communications line has higher priority than the printer, the interrupt is honored.

The printer ISR is interrupted, its state is pushed onto the stack, and execution continues at the communications ISR.

While this routine is executing, a disk interrupt occurs ($t=20$). Because this interrupt is of lower priority, it is simply held, and the communications ISR runs to completion.

Multiple Interrupts: Example

When the communications ISR is complete ($t = 25$), the previous processor state is restored, which is the execution of the printer ISR.

However, before even a single instruction in that routine can be executed, the processor honors the higher priority disk interrupt and control transfers to the disk ISR. Only when that routine is complete ($t = 35$) is the printer ISR resumed.

When that routine completes ($t = 40$), control finally returns to the user program.

Refer to the next figure

Multiple Interrupts: Example

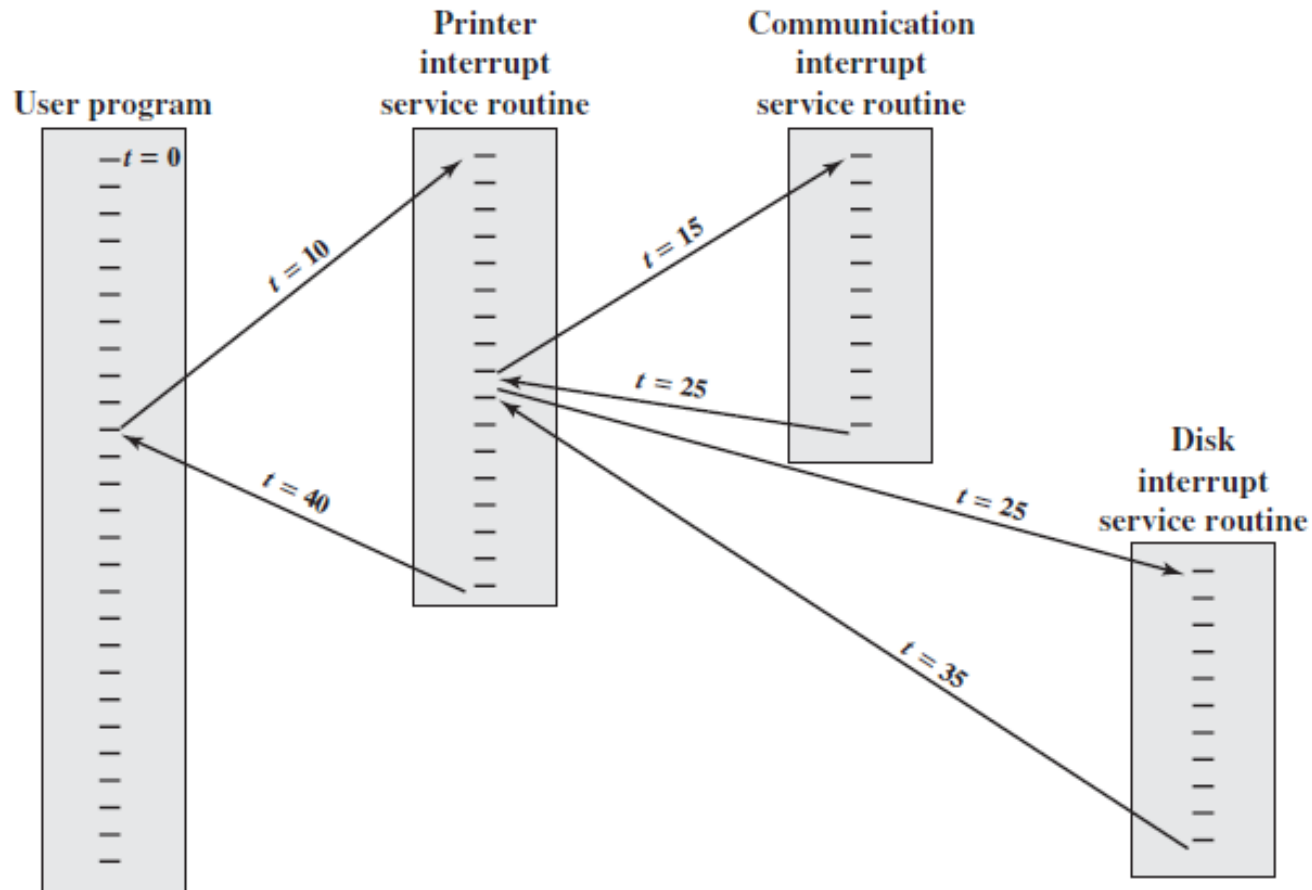


Figure 3.14 Example Time Sequence of Multiple Interrupts